

Exercise Sheet 9

Deadline for submission: Wednesday, 17 December 2025 at 8:00.

Discussed in exercise session: Friday, 19 December 2025 at 10:15.

Exercise 9.1 - Optimization without Gradient: Downhill Simplex

In this exercise, we are going to implement the downhill simplex method, also known as Nelder–Mead method. The goal is to optimize Rosenbrock function, a well-known test function in optimisation. This function is frequently used to evaluate the performance of optimization algorithms. It is sometimes called the “Rosenbrock valley” or “Rosenbrock banana function” due to its characteristic curved, narrow valley shape. The function is defined as:

$$f(x, y) = (a - x)^2 + b(y - x^2)^2, \quad (1)$$

where a and b are constants, often set to $a = 1$ and $b = 100$. This function has a global minimum at $(x, y) = (a, a^2)$, where $f(x, y) = 0$.

Please follow the instructions in the notebook and finish the implementation.

Exercise 9.2 - Dynamic Programming: Optimal Binary Search Trees (OBST)

Given a set of keys $k_1, \dots, k_n$ and their associated probabilities $p_1, \dots, p_n$ (or equivalently frequencies, representing unnormalized probabilities), determine the structure of the optimal binary search tree (OBST), that minimizes the total search cost.

index	1	2	3	4
key k_i	10	12	16	21
frequency	4	2	6	3

Answer the following questions:

1. Complete the Dynamic Programming (DP) matrix (Figure 1) by calculating the costs for all possible subtrees based on the provided keys and their frequencies.
2. What is the cost of the *optimal* search tree? Which entry indicates that in the DP matrix?
3. Using the DP matrix you completed in task 1, draw the OBST.

4. Verify that the total search cost derived from your OBST structure from Task 3 matches the optimal cost found in Task 2.

	1	2	3	4
1				
2				
3				
4				

Figure 1: DP for an OBST

Exercise 9.3 - Needleman-Wunsch Algorithm

Assume we want to align the strings

$x = \text{PAWHEAE}$ and $y = \text{HEAGAWGHEE}$

using three different operations:

- align $x(i)$ with $y(j)$ (alignment score given by substitution matrix)
- align $x(i)$ with gap in y (gap cost d)
- align $y(j)$ with gap in x (gap cost d).

In our setting, the gap cost is $d = 8$ and we use the BLOSUM50 substitution matrix. The file `scoring_Blosum50.py` contains the function `blosum50(a,b)` which returns the score for the given input pair a, b :

```
score = blosum50('A', 'H')           (result: score = -2)
```

The Needleman-Wunsch Algorithm (1970) finds the optimal global alignment between x and y . In particular, it computes...

- a score matrix F
- the score of the optimal alignment (bottom right element of F)
- a matrix of pointers
- the traceback of pointers, which gives the actual alignment.

Complete the Python code provided in the jupyter notebook. The goal is to compute and visualize the score matrix F and the optimal global alignment.

Hint: You can compare your results to the lecture slides. The values are identical. (The model solutions are also directly available in the code.)